Finding the bugs that don't crash

A smart fuzzer for PostgreSQL that found 9 real bugs

LLM-guided grammar-aware fuzzing · CS-5187 term project A3

TARGET

Our target: PostgreSQL

- Used by banks, companies and apps everywhere
- Its query planner changes almost every week, so new bugs keep appearing
- **Worst bugs are silent wrong answers: no crash, no error, just wrong rows**
- We tested the newest development version (with internal checks on) and releases 17 and 18

9

bugs reported to PostgreSQL

3 + 4

fixed + accepted / confirmed

Example of a silent wrong answer: a query asks "is this empty value inside the range?" PostgreSQL 17 and 18 answer "yes" instead of "unknown".

TECHNIQUES

How our fuzzer works

1 Generate

Writes random but valid SQL from a grammar (rules of the SQL language)

→

2 Check

Runs each query and checks the answer in several ways ("oracles")

→

3 Learn

An LLM reads new PostgreSQL code and adds new SQL features; a gate tests them first

→

4 Report

Shrinks a failing query to a few lines and drafts the bug report

How we know an answer is wrong

Crash checks

the database crashes or an internal safety check fails

Rewrite checks

ask the same question two ways; answers must match (e.g. split WHERE into true / false / unknown)

Plan & version checks

same query with different planner settings, or on the old release, must match

BUG 1 OF 6

FIXED · #19803

Parallel query planner crash

How to trigger it

```
CREATE TABLE pi (a int);
CREATE TABLE pi_child () INHERITS (pi);

WITH c1 AS (
  SELECT 1 FROM pi WHERE true
  UNION ALL
  SELECT 1 FROM pi WHERE false),
c2 AS MATERIALIZED (SELECT 1)
SELECT 1 FROM c1
UNION ALL SELECT 2 FROM c2;

-- server crashes (assertion failure)
```

What goes wrong

The planner prepares a parallel plan for a table, then marks the table "not safe for parallel" but forgets the plan it already made. An internal safety check stops the server.

How we caught it

The LLM part of our fuzzer: grammar rules it learned from a recent PostgreSQL change produced this query; the crash check caught it.

What PostgreSQL did

Confirmed and fixed by David Rowley (commit abda2a2c), back-ported to version 14.

BUG 2 OF 6

FIXED · OUR PATCH · #19771

to_char() silently drops part of its output

How to trigger it

```
SELECT to_char(100::numeric,
               'FM999.SG "items"');

-- before the fix:   100
-- after  the fix:   100+ items

-- any number format that ends in a
-- decimal point loses everything
-- after it
```

What goes wrong

When a number format ends with a decimal point, PostgreSQL copies an end-of-string byte into the output, so the sign and any text after it disappear. No error is shown.

How we caught it

Our cross-version check: we ran thousands of random to_char / to_number / to_date calls on two PostgreSQL versions and compared the answers.

What PostgreSQL did

Our team wrote the fix; Tom Lane reviewed and committed it (776df8a6) for version 19.

BUG 3 OF 6

FIXED · OUR PATCH

Statistics on expressions crash the planner

How to trigger it

```
CREATE TABLE dt (a int, c bool);
INSERT INTO dt SELECT x % 10, x % 10 < 5
  FROM generate_series(1, 1000) x;
CREATE STATISTICS s (dependencies)
  ON a, (c IS TRUE) FROM dt;
ANALYZE dt;

SELECT * FROM dt
WHERE a = 1 AND (c IS TRUE)
            AND (c IS TRUE);

-- server crashes (assertion failure)
```

What goes wrong

With extra statistics on an expression, writing the same condition twice makes the planner's estimate code trip an internal safety check that contradicts its own logic.

How we caught it

Our statistics fuzzer: it builds random statistics objects and random WHERE clauses that reuse the same expressions; the crash check caught it.

What PostgreSQL did

Our team wrote the fix; David Rowley reviewed and committed it (b0392c26), back-ported to version 14.

BUG 4 OF 6

ACCEPTED · #19768

JSON_TABLE gives wrong answers

How to trigger it

```
SELECT * FROM JSON_TABLE(
  jsonb_build_object('k',
      ROW(1, 2) < ROW(3, 4)),
  '$' COLUMNS (a bool PATH '$.k'));

-- expected:   t     (1,2) < (3,4) is true
-- 17 and 18:  f     wrong answer
-- newest:     server crashes
```

What goes wrong

JSON_TABLE turns JSON into rows. It forgets to set up text-comparison rules (collations) for its input, so comparing rows inside it gives false, or a bogus error.

How we caught it

The grammar fuzzer generated a JSON_TABLE query that crashed the newest version; checking the same query on release 18 showed the silent wrong answer.

What PostgreSQL did

Accepted upstream; our one-line fix with tests was sent.

BUG 5 OF 6

ACCEPTED

NOT IN returns rows it should not

How to trigger it

```
-- a cast that turns 0 into NULL
CREATE CAST (int AS text)
  WITH FUNCTION i2t(int) AS IMPLICIT;

CREATE TABLE o (t text NOT NULL);  -- '1'
CREATE TABLE s (i int  NOT NULL);  --  0

SELECT t FROM o
WHERE t NOT IN (SELECT i FROM s);

-- expected: no rows  ('1' vs NULL is unknown)
-- actual:   1
```

What goes wrong

A new speed-up in version 19 turns NOT IN into a faster join. It assumes the values can never be NULL, but a conversion (cast) can create NULLs, so wrong rows come back.

How we caught it

We reviewed the new speed-up and checked it against other plans and the older release; our rewrite check also flags it (one row vs two).

What PostgreSQL did

Accepted upstream; our fix with a test was sent.

BUG 6 OF 6

CONFIRMED · #19802

MERGE fails with an internal error

How to trigger it

```
CREATE TABLE mp (a int, b text)
  PARTITION BY RANGE (a);
CREATE TABLE mp1 PARTITION OF mp
  FOR VALUES FROM (0) TO (10);

MERGE INTO mp t
USING (SELECT true AS c OFFSET 0) s
  ON t.a IS NULL
WHEN MATCHED THEN UPDATE
  SET b = (SELECT NULL WHERE s.c)
WHEN NOT MATCHED THEN
  INSERT VALUES (1, 'x');

-- ERROR: SubPlan found with no parent plan
```

What goes wrong

MERGE (insert-or-update in one statement) on a partitioned table, with a small subquery in the UPDATE, makes the planner lose track of that subquery: a valid statement fails with an internal error.

How we caught it

The grammar fuzzer generated it; the internal-error check flags any "this should never happen" error.

What PostgreSQL did

Confirmed by PostgreSQL committer Tender Wang, who posted a fix.

SUMMARY

All results

Bug	Kind	Status
Parallel planner crash (#19803)	crash	**fixed**
to_char() drops output (#19771)	wrong output	**fixed (our patch)**
Statistics crash (dependencies.c)	crash	**fixed (our patch)**
JSON_TABLE wrong answer (#19768)	wrong answer	**accepted**
NOT IN wrong rows	wrong answer	**accepted**
MERGE internal error (#19802)	internal error	**confirmed**
IGNORE NULLS query never ends	hang	**confirmed**
Range check wrong for NULL (#19793)	wrong answer	**reported**
Parallel EXPLAIN crash (#19766)	crash	**reported**

3

fixed in PostgreSQL (2 with our patches)

4

accepted / confirmed

Takeaway: writing valid SQL is easy; knowing the right answer is the hard part.

Thank you! Questions?

Code, reproducers and patches: github.com/PASS-IITG/cs5187-term-project-blazing-bhuvi